# Supplementary Material for "TempFlash: GPU-Resident, Single-Launch Training of Temporal Graph Neural Networks"

Weiwei Ye
The University of Tokyo
Japan
wwye155@gmail.com

Zhen Wang
Huzhou Normal University
China
zhenwangrs@gmail.com

Yankai Chen
Mohamed bin Zayed University of Artificial Intelligence
United Arab Emirates
yankaichan3@gmail.com

Zhen Bi
Huzhou Normal University
China
bizhen_zju@zju.edu.cn

Dongyuan Li
The University of Tokyo
Japan
lidy@csis.u-tokyo.ac.jp

*This document is the supplementary material of the paper. References to Sections I–VII, Tables I–III, Figures 1–8 and Equations (1)–(7) point to the paper; the bibliography for this document appears at its end.*

## A PROOF OF PROPOSITION 1

**Reference semantics.** Consider TGL's recent sampler with a single history and layer, the configuration used here. Node $v$'s non-decreasing timestamps occupy T-CSR segment $[o_v, o_{v+1})$, and reset initializes $p_v$ to $o_v$. For roots $(v_r, t_r)$, $r = 1, \ldots, R$, with fp32 thresholds $\theta_r = \mathrm{fl}(t_r - 10^{-7})$, all pointer updates precede all root sampling. Updating root $r$ scans $j = p_{v_r}, p_{v_r} + 1, \ldots$ to the first $j$ satisfying $ts_j > \theta_r$; when $j \neq p_{v_r}$ it assigns $p_{v_r} \leftarrow j - 1$, otherwise retains the pointer, or assigns $o_{v_r+1} - 1$ upon reaching the segment's end. A per-node lock serializes updates in an arbitrary order. To sample root $r$, entries $k = p_{v_r}, p_{v_r} - 1, \ldots$ are visited down to but not including $\max(o_{v_r}, p_{v_r} - K)$; those satisfying $ts_k < \theta_r$ are emitted in visitation order, from newest to oldest.

**Lemma 1 (one update).** *Define $u_r$ as the first index in $v_r$'s segment with timestamp greater than $\theta_r$, or $o_{v_r+1}$ if no such entry exists. Updating root $r$ replaces $p_{v_r}$ by* $\max(p_{v_r}, u_r - 1)$. Since the segment is sorted, timestamps exceeding $\theta_r$ occupy its suffix $[u_r, o_{v_r+1})$. For $u_r > p_{v_r}$, scanning either stops at $j = u_r \neq p_{v_r}$ or reaches the segment boundary when $u_r = o_{v_r+1}$, giving $u_r - 1$ in both cases. For $u_r \leq p_{v_r}$, the first entry stops the scan without changing the pointer; an empty segment performs no scan, with $u_r - 1 < o_{v_r} = p_{v_r}$. Thus all cases yield $\max(p_{v_r}, u_r - 1)$.

**Lemma 2 (order independence).** *Regardless of update order, the batch leaves* $p_v = \max\left(p_v^{\mathrm{old}}, \max_{r:v_r=v}(u_r - 1)\right)$. Lemma 1 expresses each update as $x \mapsto \max(x, c_r)$; these maps commute, and composing them gives $x \mapsto \max(x, \max_r c_r)$, precisely the batch pointer rule in Eq. (4).

**Lemma 3 (search).** *Searching the key array for the upper bound of $(v_r \ll 32) \mid \varphi(\theta_r)$ returns $u_r$.* Equation (2)'s $\varphi$ follows floating-point ordering for non-NaN fp32 values apart from putting $-0$ below $+0$; neither timestamps shifted to start at zero nor thresholds from fp32 subtraction take the value $-0$. Consequently, for node $v_r$'s entry $i$, the condition that $\kappa_i$ exceeds the query is equivalent to $ts_i > \theta_r$, while other nodes' entries compare according to node identifier. The first greater key in this sorted array is therefore at $u_r$.

**Table IV: State commits: frequency of retaining an update other than the final occurrence (CPU: counts; GPU and Write: percentage of repeated nodes).**

| Data | Model | Seq. | Batches | Updates | Dup. | CPU | GPU | Write |
|---|---|---|---|---|---|---|---|---|
| LastFM | TGN | mails | 768 | 1.81M | 296.2K | 0 | 55.9% | 57.1% |
| LastFM | TGN | memories | 768 | 1.81M | 296.2K | 0 | 57.9% | 58.6% |
| LastFM | APAN | mails | 768 | 11.70M | 678.0K | 0 | 80.8% | 81.2% |
| Wiki-Talk | TGN | mails | 2,589 | 10.97M | 1.64M | 0 | 66.3% | 67.0% |
| Wiki-Talk | TGN | memories | 2,589 | 10.97M | 1.64M | 0 | 57.4% | 58.5% |
| Wiki-Talk | APAN | mails | 2,589 | 65.34M | 6.33M | 0 | 77.9% | 78.1% |
| Stack-Ovf. | TGN | mails | 300 | 1.49M | 243.1K | 0 | 61.8% | 62.3% |
| Stack-Ovf. | TGN | memories | 300 | 1.49M | 243.1K | 0 | 63.8% | 64.5% |
| GDELT | TGN | mails | 300 | 1.58M | 148.4K | 0 | 79.7% | 80.0% |
| GDELT | TGN | memories | 300 | 1.58M | 148.4K | 0 | 80.1% | 80.6% |

**Proof of Proposition 1.** The GPU computes $\theta_r$ using the identical fp32 subtraction. Applying Lemmas 2 and 3 inductively over batches after reset shows that maximum-reduction scatter produces the same pointers as the reference after each update phase. With $p = p_{v_r}$ and $m = \min(p - o_{v_r}, K)$, the reference emits entries $p - j$ for $j = 0, \ldots, m - 1$ whose timestamps lie below $\theta_r$, preserving that order. GPU slot $j$ accesses $p - j$ and applies exactly these conditions, so reading valid slots rowwise recovers the reference sequence. Both read neighbor identifiers, event identifiers and timestamps from identical entries and calculate deltas through the same fp32 subtraction. □

## B CORRECTNESS TESTS

**Sampler.** In addition to the four Table I graphs (§IV-D), we tested the GPU sampler against TGL's C++ sampler on 2,000 random graphs designed to frequently exercise corner cases: few distinct times, many ties, repeated roots, nodes with no or little prior history, and thresholds equal to stored timestamps. Each graph undergoes two passes separated by a reset, using $K \in \{1, 2, 3, 10\}$ and either one or eight sampler threads. The 41,404 batches contain 1.12M roots and 963.6K sampled neighbors, including 397,571 threshold-equal entries, 38,789 repeated-root batches, 248,918 excluded first entries, 16,752 empty histories and 431,282 short histories; across them, neighbor, event-identifier, timestamp and ordering differences total 0.

**Commit semantics.** We constructed each batch's candidate sequence in reference order (TGN: event-interleaved mails, source

**Table V: Training time per epoch (s) for the TGAT and APAN ports of Orca and Zebra, alongside TempFlash (Table II).**

| | LastFM | | Wiki-Talk | | Stack-Overflow | | GDELT | |
|---|---|---|---|---|---|---|---|---|
| | TGAT | APAN | TGAT | APAN | TGAT | APAN | TGAT | APAN |
| Orca | 9.91 | 14.5 | 72.1 | 228.2 | 1390.7 | 2006.8 | 5095.3 | 1244.6 |
| Zebra | 36.6 | 41.6 | 212.4 | 418.5 | 2942.1 | 3208.6 | 9837.5 | 4900.3 |
| TempFlash | 1.64 | 3.49 | 13.2 | 15.7 | 126.0 | 159.2 | 330.8 | 347.3 |

memories followed by destination memories; APAN: root mails followed by mails to each root's sampled neighbors) and checked last-occurrence agreement for ETC selection on CPU, selection on GPU, and plain GPU indexed writes without selection. Table IV records, per sequence, inspected batches (300 strided across Stack-Overflow and GDELT epochs), update counts and nodes repeated within a batch, summed across batches. By construction, TempFlash always commits each node's final occurrence (§III-E).

**Differential test.** We initialize TempFlash from an ETC model instance and train it alongside the last-writer reference variant with identical features, adaptive batches and negative destinations, disabling dropout. After each batch, comparisons cover relative prediction and loss differences, every post-Adam parameter, absolute differences in touched-node memories and mails, and exact equality of memory timestamps, mail timestamps and ring pointers. CPU selection was observed to retain the final occurrence (Table IV). Across all 768 LastFM training batches, TGN and APAN discrete states match exactly, with 0 differing timestamps or pointers. Re-synchronizing TempFlash to reference state before each batch bounds maximum relative differences across 300 batches of all three models at $1.6 \times 10^{-6}$ for predictions and $2.4 \times 10^{-6}$ for parameters.

The full-step comparison concerns the tested last-writer reference variant, not unchanged released ETC. Dropout is disabled; aligned-mask stochastic-step equivalence is not tested.

**Rounding against fp64.** An fp64 reference-model copy evaluates identical neighborhoods and pre-step states for 6 synchronized TGN batches. Median relative fp32 prediction errors are $3.5 \times 10^{-7}$ for TempFlash and $4.6 \times 10^{-7}$ for the reference; maximum relative gradient errors are $8.8 \times 10^{-7}$ and $2.2 \times 10^{-5}$, respectively, so TempFlash introduces no greater rounding error.

## C BASELINES

**Orca and Zebra.** Their releases build on TGN's implementation [1] and support TGN alone. We added ports for the other two models. TGAT is obtained as the memory-free case: removing node memory and using node features at layer 0 while retaining two attention layers, Orca's reused embeddings and Zebra's PageRank neighborhoods. Our APAN port follows its default configuration: ten mails per node, TGL's attention memory updater with layer normalization for embeddings, and mail delivery to each node and its system-sampled neighbors (Orca's ten most recent, or Zebra's top-$k$). Port epoch times in Table V are 3.6–29.7 times TempFlash's.

## D ADDITIONAL DYNAMIC GRAPHS

**Accuracy and speed.** Table VI reports three-seed test AP and epoch duration for TempFlash and ETC on the 36 workloads introduced in §IV-I. Their average AP gap is 0.69 points; on 35 workloads it remains within twice the larger seed standard deviation, or one point. Back-to-back timing against ETC on a dedicated GPU gives TempFlash a 3.7–8.3× per-epoch speedup across all 36 workloads, with a geometric mean of 5.2× over these graph–model combinations.

**Algorithm 1** Training step captured for size bucket $B$

**Require:** cursor $c$; batch starts $lo[\cdot]$ and sizes $sz[\cdot]$; event arrays; T-CSR; tiers; states

1: $n \leftarrow sz[c]$; $ok \leftarrow [0, B) < n$; $ev \leftarrow \min(lo[c] + [0, B),\ |E| - 1)$ ▷ device cursor
2: $(\mathbf{src}, \mathbf{dst}, \mathbf{t}) \leftarrow \text{events}[ev]$; $\mathbf{neg} \leftarrow \text{randint}(|V|, B)$
3: roots $\leftarrow [\mathbf{src}; \mathbf{dst}; \mathbf{neg}]$ at times $[\mathbf{t}; \mathbf{t}; \mathbf{t}]$
4: $(nbr, eid, \Delta t, mask) \leftarrow \textsc{Sample}(\text{roots}, ok)$ ▷ §III-B
5: read features, memories and mails into slots ▷ §III-D, §III-F
6: $\mathcal{L} \leftarrow \textsc{MaskedBCE}(\textsc{Model}(\cdot), ok)$; $\nabla\mathcal{L}$; Adam step ▷ §III-C
7: $\textsc{Commit}$(new memories and mails, $ok$) ▷ §III-E
8: $c \leftarrow c + 1$

## E IMPLEMENTATION DETAILS

**Captured step.** Algorithm 1 lists the training step that TempFlash captures for one batch-size bucket and replays once per batch (§III-G).

**Sampling time.** Table III measures one epoch's training-batch sampling with TGL's C++ implementation on 1, 8 and 32 host threads and TempFlash's GPU sampler using either individual launches (eager) or captured replay. Identical roots produce identical neighbors in both samplers; Nbrs. lists the epoch's total neighbor count. Captured GPU sampling outpaces eight-thread TGL by 9.9–17× and 32-thread TGL by 10–29×.

**Memory.** Table VII accounts for TempFlash GPU storage by T-CSR (Entries gives its entry count; T-CSR includes sorting keys), event arrays, node and edge features, node states (memories and mailboxes), and remaining tensors including step workspace; Host records the edge-feature table held in pinned, zero-copy host memory.

**Step anatomy.** With all data resident, a training step issues 344–485 kernels (Table VIII); captured replay avoids their individual host dispatches and improves step time by up to 1.9×, with the greatest benefit for small-batch memory-based models.

**Table VI: Results on additional graphs: seed counts, test AP (%, mean ± standard deviation), epoch time (s), and speedup.**

| Data | Model | $n$ | ETC AP | TempFlash AP | ETC | TempFlash | Spd. |
|---|---|---|---|---|---|---|---|
| Wikipedia | TGN | 3 | 97.53 ± 0.27 | 97.30 ± 0.19 | 3.14 | 0.84 | 3.8× |
| | TGAT | 3 | 88.64 ± 0.21 | 88.62 ± 0.25 | 2.46 | 0.37 | 6.6× |
| | APAN | 3 | 91.07 ± 0.65 | 91.01 ± 0.27 | 2.87 | 0.50 | 5.7× |
| Reddit | TGN | 3 | 99.14 ± 0.63 | 99.50 ± 0.05 | 18.1 | 3.72 | 4.9× |
| | TGAT | 3 | 96.36 ± 0.25 | 96.09 ± 0.12 | 7.86 | 1.45 | 5.4× |
| | APAN | 3 | 98.52 ± 0.60 | 98.75 ± 0.10 | 12.1 | 1.89 | 6.4× |
| MOOC | TGN | 3 | 99.34 ± 0.04 | 99.37 ± 0.04 | 6.78 | 1.69 | 4.0× |
| | TGAT | 3 | 97.13 ± 0.08 | 97.14 ± 0.02 | 5.17 | 0.72 | 7.1× |
| | APAN | 3 | 98.87 ± 0.06 | 98.89 ± 0.09 | 5.39 | 0.98 | 5.5× |
| Enron | TGN | 3 | 71.88 ± 2.02 | 70.84 ± 1.97 | 2.73 | 0.55 | 4.9× |
| | TGAT | 3 | 70.75 ± 0.67 | 70.65 ± 0.40 | 1.95 | 0.31 | 6.3× |
| | APAN | 3 | 52.99 ± 1.54 | 53.58 ± 1.96 | 1.85 | 0.33 | 5.6× |
| SocialEvo | TGN | 3 | 80.56 ± 0.05 | 82.46 ± 0.31 | 38.1 | 7.69 | 5.0× |
| | TGAT | 3 | 77.94 ± 0.07 | 77.90 ± 0.02 | 23.4 | 3.52 | 6.7× |
| | APAN | 3 | 63.96 ± 4.86 | 60.77 ± 9.76 | 23.9 | 4.32 | 5.5× |
| UCI | TGN | 3 | 80.15 ± 3.91 | 76.86 ± 7.22 | 1.29 | 0.33 | 3.9× |
| | TGAT | 3 | 63.11 ± 1.25 | 63.17 ± 0.17 | 0.97 | 0.20 | 4.8× |
| | APAN | 3 | 62.61 ± 2.17 | 62.47 ± 1.13 | 1.08 | 0.21 | 5.1× |

| Data | Model | $n$ | ETC AP | TempFlash AP | ETC | TempFlash | Spd. |
|---|---|---|---|---|---|---|---|
| Flights | TGN | 3 | 90.38 ± 1.63 | 90.85 ± 1.66 | 32.5 | 8.43 | 3.9× |
| | TGAT | 3 | 82.12 ± 0.08 | 82.08 ± 0.06 | 19.6 | 3.33 | 5.9× |
| | APAN | 3 | 87.35 ± 0.36 | 88.01 ± 1.46 | 24.0 | 4.43 | 5.4× |
| CanParl | TGN | 3 | 72.09 ± 3.34 | 72.85 ± 1.44 | 3.06 | 0.37 | 8.3× |
| | TGAT | 3 | 71.18 ± 0.22 | 71.26 ± 0.24 | 0.93 | 0.19 | 5.0× |
| | APAN | 3 | 63.41 ± 0.67 | 64.76 ± 2.09 | 0.98 | 0.22 | 4.5× |
| USLegis | TGN | 3 | 74.58 ± 0.68 | 75.88 ± 0.29 | 1.29 | 0.25 | 5.1× |
| | TGAT | 3 | 68.08 ± 0.33 | 68.05 ± 0.59 | 0.85 | 0.13 | 6.4× |
| | APAN | 3 | 65.66 ± 4.88 | 64.16 ± 3.98 | 0.81 | 0.16 | 5.1× |
| UNtrade | TGN | 3 | 65.43 ± 0.59 | 65.58 ± 0.86 | 7.46 | 1.89 | 4.0× |
| | TGAT | 3 | 53.37 ± 0.05 | 53.38 ± 0.04 | 5.63 | 0.87 | 6.5× |
| | APAN | 3 | 60.01 ± 1.62 | 60.90 ± 3.32 | 5.90 | 1.09 | 5.4× |
| UNvote | TGN | 3 | 58.56 ± 1.41 | 57.96 ± 0.35 | 14.1 | 3.84 | 3.7× |
| | TGAT | 3 | 50.93 ± 0.01 | 50.95 ± 0.03 | 11.0 | 1.71 | 6.4× |
| | APAN | 3 | 55.60 ± 1.27 | 55.28 ± 1.10 | 10.9 | 2.18 | 5.0× |
| Contacts | TGN | 3 | 89.97 ± 0.17 | 89.56 ± 1.12 | 42.1 | 10.4 | 4.1× |
| | TGAT | 3 | 88.45 ± 0.08 | 88.38 ± 0.10 | 28.0 | 4.59 | 6.1× |
| | APAN | 3 | 87.05 ± 1.43 | 82.48 ± 5.29 | 24.7 | 5.83 | 4.2× |

**Table VII: Array-level GPU allocation (GB) for each TempFlash workload, together with edge features in zero-copy host storage. $^{p}$ denotes bit packing; $^{f}$ denotes mailbox factorization.**

| Data | Model | Entries | T-CSR | Events | Node feat. | Edge feat. | States | Other | Total | Host |
|---|---|---|---|---|---|---|---|---|---|---|
| LastFM | TGN | 2.59M | 0.05 | 0.01 | 0.00 | 0.62 | 0.00 | 0.12 | 0.80 | 0.0 |
| LastFM | TGAT | 2.59M | 0.05 | 0.01 | 0.00 | 0.62 | 0.00 | 0.09 | 0.77 | 0.0 |
| LastFM | APAN | 2.59M | 0.05 | 0.01 | 0.00 | 0.62 | 0.03 | 0.10 | 0.80 | 0.0 |
| Wiki-Talk | TGN | 7.83M | 0.16 | 0.09 | 0.73 | 5.02 | 2.03 | 0.17 | 8.20 | 0.0 |
| Wiki-Talk | TGAT | 7.83M | 0.16 | 0.09 | 0.73 | 5.02 | 0.00 | 0.13 | 6.13 | 0.0 |
| Wiki-Talk | APAN | 7.83M | 0.16 | 0.09 | 0.00 | 5.02 | $9.07^{f}$ | 0.13 | 14.47 | 0.0 |
| Stack-Ovf. | TGN | 63.50M | 1.22 | 0.71 | 1.67 | 0.00 | 4.63 | 0.27 | 8.50 | 40.7 |
| Stack-Ovf. | TGAT | 63.50M | 1.22 | 0.71 | 1.67 | 0.00 | 0.00 | 0.21 | 3.81 | 40.7 |
| Stack-Ovf. | APAN | 63.50M | 1.22 | 0.71 | 0.00 | 0.00 | $20.70^{f}$ | 0.22 | 22.85 | 40.7 |
| GDELT | TGN | 191.29M | 3.56 | 2.14 | 0.03 | $4.10^{p}$ | 0.03 | 0.39 | 10.24 | 0.0 |
| GDELT | TGAT | 191.29M | 3.56 | 2.14 | 0.03 | $4.10^{p}$ | 0.00 | 0.37 | 10.19 | 0.0 |
| GDELT | APAN | 191.29M | 3.56 | 2.14 | 0.03 | $4.10^{p}$ | 0.24 | 0.35 | 10.42 | 0.0 |

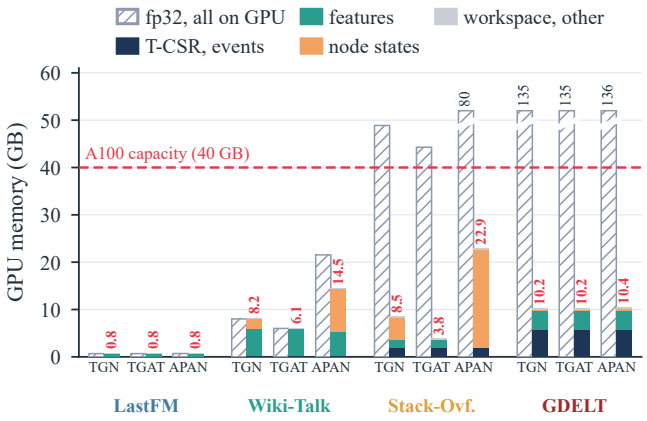



**Figure 9: Per-workload GPU footprint under full fp32 storage (hatched) and TempFlash allocation (solid).**

**Table VIII: Kernel counts and wall-clock step times with default settings and fully resident data: individual kernel launches (Eager) versus captured replay (Replay).**

| Data | Model | Kernels | Eager (ms) | Replay (ms) | Ratio |
|---|---|---|---|---|---|
| LastFM | TGN | 474 | 8.0 | 6.0 | 1.3× |
| | TGAT | 349 | 10.5 | 9.7 | 1.1× |
| | APAN | 377 | 6.6 | 3.4 | 1.9× |
| Wiki-Talk | TGN | 468 | 13.0 | 12.4 | 1.0× |
| | TGAT | 345 | 15.0 | 14.5 | 1.0× |
| | APAN | 392 | 6.7 | 6.1 | 1.1× |
| Stack-Ovf. | TGN | 470 | 17.9 | 17.3 | 1.0× |
| | TGAT | 344 | 19.1 | 18.6 | 1.0× |
| | APAN | 393 | 10.0 | 9.4 | 1.1× |
| GDELT | TGN | 485 | 13.4 | 12.7 | 1.1× |
| | TGAT | 350 | 11.5 | 11.0 | 1.0× |
| | APAN | 379 | 7.6 | 7.0 | 1.1× |

## REFERENCES

[1] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. *arXiv preprint arXiv:2006.10637* (2020).